

Sistema de Radio Definida por Software (SDR) utilizando ADALM-Pluto y Python

Proyecto de Electrónica II

Integrantes:

Jeniffer Baculima

Jhoselyn Auquilla

Edgar Magi

Rommerl Soto

Asignatura: Electrónica II

Docente: Geovanny Cuzco

Institución: Universidad Nacional de Chimborazo

Fecha: 15 Abril 2026

Capítulo 1

PROYECTO PRIMER PARCIAL

Objetivo del PROYECTO

Implementar un sistema de Radio Definida por Software (SDR) utilizando la plataforma ADALM-Pluto y el lenguaje de programación Python, con el fin de analizar señales en banda base mediante procesamiento digital, visualización de componentes I/Q y análisis espectral mediante FFT.

Objetivos Específicos:

- Configurar la plataforma ADALM-Pluto para transmisión y recepción de señales mediante Python.
- Generar señales de prueba (senoidales) en banda base para validación del sistema SDR.
- Capturar muestras I/Q provenientes del receptor para su análisis en tiempo y frecuencia.
- Implementar el cálculo de la Transformada Rápida de Fourier (FFT) para el análisis espectral de las señales recibidas.
- Visualizar las señales procesadas utilizando herramientas como matplotlib.
- Establecer la base para la implementación futura de modulaciones analógicas como AM.

1.1. Introducción

La Radio Definida por Software (SDR) es una tecnología que permite implementar sistemas de comunicación mediante software, reemplazando componentes tradicionales de hardware por procesamiento digital de señales. Esto permite una mayor flexibilidad en el diseño, análisis y adaptación de sistemas de comunicación modernos.

El uso de SDR es ampliamente relevante en ingeniería electrónica debido a que permite el estudio práctico de señales de radiofrecuencia sin la necesidad de circuitos analógicos complejos, facilitando la experimentación y el aprendizaje de conceptos fundamentales como modulación, demodulación y análisis espectral.

En este contexto, la plataforma ADALM-Pluto, desarrollada por Analog Devices, se utiliza como herramienta principal debido a su capacidad de transmisión y recepción en un amplio

rango de frecuencias mediante el transceptor AD9363. Su bajo costo y compatibilidad con herramientas de software la convierten en una solución ideal para entornos educativos.

Asimismo, el lenguaje de programación Python es empleado debido a su facilidad de uso y la disponibilidad de librerías especializadas como pyadi-io, NumPy y Matplotlib, las cuales permiten el procesamiento y visualización de señales en tiempo real.

El objetivo de este proyecto es implementar un sistema de Radio Definida por Software (SDR) que permita la generación, transmisión y análisis de señales en banda base, así como la captura de componentes en cuadratura (I/Q) y su análisis espectral mediante la Transformada Rápida de Fourier (FFT). Este enfoque permite sentar las bases para la implementación de técnicas de modulación analógica en etapas posteriores.

Capítulo 2

Marco Teórico

2.1. Radio Definida por Software (SDR)

2.1.1. Fundamentos de Radio Definida por Software (SDR)

La Radio Definida por Software (SDR) representa un cambio de paradigma en la ingeniería de telecomunicaciones, donde las funciones tradicionalmente ejecutadas por componentes de hardware dedicados (como mezcladores, filtros, amplificadores, moduladores y demoduladores) se implementan mediante software en dispositivos de computación programables [1].

Arquitectura y Evolución Tecnológica

Históricamente, los sistemas de radio se basaban en la arquitectura superheterodina, la cual depende de múltiples etapas de mezcla analógica y filtrado de frecuencia intermedia (IF) rígido. En contraste, un sistema SDR simplifica la cadena de RF al desplazar el límite de la digitalización lo más cerca posible de la antena [2]. Esta flexibilidad permite que un solo hardware físico sea reconfigurado para operar en diferentes bandas de frecuencia y soportar diversos estándares de comunicación (AM, FM, LTE, 5G, etc.) simplemente actualizando el código de procesamiento.

Transceptores de Conversión Directa (Zero-IF)

La plataforma ADALM-Pluto, núcleo de este proyecto, utiliza una arquitectura de conversión directa o *Zero-IF*. Según Stewart et al. [3], en esta topología la señal de RF entrante se mezcla directamente con una señal de oscilador local (LO) a la misma frecuencia central, traduciendo el espectro directamente a la banda base (0 Hz).

A diferencia de las radios tradicionales, el diseño *Zero-IF* elimina la necesidad de filtros de rechazo de imagen costosos, permitiendo una integración masiva en un solo chip de silicio (System-on-Chip). Sin embargo, esta arquitectura introduce desafíos técnicos específicos que deben ser gestionados por software:

- **Desplazamiento de DC (DC Offset):** El auto-mezclado del oscilador local genera una componente de tensión continua en el centro del espectro.

- **Ruido de Parpadeo ($1/f$):** Ruido electrónico que predomina en bajas frecuencias y puede degradar la señal de información.
- **Desbalance I/Q:** Pequeñas diferencias de ganancia o fase entre las ramas real e imaginaria que pueden comprometer la integridad de la constelación o la demodulación [4].

El Rol de la FPGA y el Host en el SDR

En el ADALM-Pluto, el procesamiento se divide entre una FPGA Zynq-7000 y un procesador host (PC). La FPGA se encarga de las tareas de alta velocidad y baja latencia, como el filtrado de diezmado y la interpolación, mientras que el software en el host (Python) gestiona la lógica de la aplicación y la visualización de datos [5]. Esta división de trabajo es esencial para mantener tasas de muestreo en tiempo real sin saturar el bus de comunicación USB.

2.1.2. Teoría de la Señal Compleja y Representación I/Q

La representación de señales mediante componentes en fase (I) y en cuadratura (Q) es el pilar fundamental que permite a los procesadores digitales manipular señales de banda estrecha de manera eficiente. En lugar de procesar la señal de RF directamente como una función sinusoidal de alta frecuencia, se utiliza la representación de envolvente compleja.

Definición Matemática de la Envolvente Compleja

Cualquier señal de radiofrecuencia pasabanda $s(t)$ puede expresarse en términos de sus componentes ortogonales mediante la siguiente identidad:

$$s(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t) \quad (2.1)$$

donde f_c es la frecuencia portadora, $I(t)$ es la componente en fase (*In-phase*) y $Q(t)$ es la componente en cuadratura (*Quadrature*), la cual presenta un desfase de 90° respecto a la primera [6].

En el dominio digital, esta pareja de señales reales se trata como una única señal analítica compleja $\tilde{s}(t)$:

$$\tilde{s}(t) = I(t) + jQ(t) \quad (2.2)$$

Esta notación permite que operaciones complejas de hardware analógico, como la modulación de fase o frecuencia, se simplifiquen a multiplicaciones y sumas de números complejos en el software [7].

Ventajas del Procesamiento I/Q en Sistemas SDR

El uso de señales I/Q en el ADALM-Pluto ofrece ventajas técnicas determinantes para el diseño de ingeniería:

- **Eliminación de Frecuencias Negativas:** Al trabajar con señales complejas, el espectro no es simétrico respecto al origen, lo que permite duplicar el ancho de banda

efectivo procesado sin ambigüedad espectral.

- **Cálculo de Magnitud y Fase:** La amplitud instantánea $A(t)$ y la fase $\phi(t)$ de la señal modulada pueden obtenerse directamente mediante coordenadas polares:

$$A(t) = \sqrt{I^2(t) + Q^2(t)}, \quad \phi(t) = \arctan\left(\frac{Q(t)}{I(t)}\right) \quad (2.3)$$

- **Independencia de la Portadora:** El procesamiento se realiza en "banda base", lo que significa que el software no necesita operar a la frecuencia de GHz de la antena, sino a la frecuencia del mensaje, reduciendo drásticamente la carga computacional [8].

Mezcla de Cuadratura y Generación de Señal

En la etapa de transmisión del AD9363, las componentes digitalizadas I y Q pasan por convertidores digital-analógicos (DAC) independientes. Posteriormente, se multiplican por dos señales de un oscilador local que están desfasadas 90° entre sí y se suman físicamente para reconstruir la señal de RF analógica. Según Haykin [9], este proceso garantiza que no existan bandas laterales no deseadas, siempre que el balance de amplitud y fase entre las ramas I y Q sea perfecto.

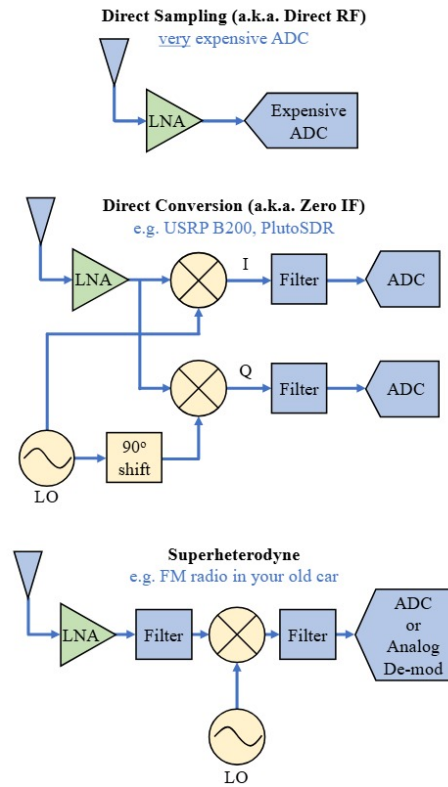


Figura 2.1: Diagrama conceptual del procesamiento de señales en fase (I) y cuadratura (Q) en la cadena digital de un transceptor SDR.

2.1.3. Conversión de Datos y Teoría del Muestreo

La integridad de cualquier sistema de comunicación digital depende de la transición precisa entre los dominios analógico y digital. En el ADALM-Pluto, este proceso es gestionado por convertidores de datos de alta velocidad integrados en el chip AD9363, operando bajo leyes matemáticas estrictas que definen la resolución y el ancho de banda del sistema.

Teorema de Nyquist-Shannon y Aliasing

El Teorema de Muestreo establece que para reconstruir una señal de banda limitada sin pérdida de información, la frecuencia de muestreo f_s debe ser estrictamente mayor al doble del ancho de banda B de la señal de interés ($f_s > 2B$) [10].

Si no se cumple esta condición, ocurre el fenómeno de *aliasing*, donde las componentes de alta frecuencia se "traslapan" sobre las bajas frecuencias, corrompiendo irreversiblemente el mensaje. En el contexto de este proyecto, dado que el ADALM-Pluto permite tasas de muestreo de hasta 61,44 MSPS, es posible procesar señales de gran ancho de banda, siempre que los filtros analógicos internos de la etapa RF limiten el espectro antes de la digitalización para cumplir con el criterio de Nyquist.

Cuantización y Resolución de Bits

Mientras que el muestreo discretiza el tiempo, la cuantización discretiza la amplitud. El AD9363 utiliza convertidores ADC y DAC de 12 bits, lo que significa que cada muestra de voltaje se mapea a uno de $2^{12} = 4096$ niveles discretos.

Esta aproximación introduce un error inevitable conocido como ruido de cuantización. La relación señal a ruido máxima teórica (SQNR) para una señal senoidal pura, asumiendo una distribución de error uniforme, se calcula mediante la ecuación fundamental de Proakis [8]:

$$SQNR_{dB} \approx 6,02N + 1,76 \text{ [dB]} \quad (2.4)$$

Para $N = 12$, el SQNR teórico es de aproximadamente 74 dB. No obstante, en condiciones de laboratorio, este valor se ve reducido por el ruido térmico, las no linealidades de los amplificadores y el *jitter* del reloj de muestreo [11].

Filtros de Diezmado e Interpolación

Para optimizar el flujo de datos hacia el host (PC), el ADALM-Pluto emplea etapas de filtrado digital. En la recepción (RX), se utiliza el diezmado para reducir la tasa de datos sin perder información relevante, mientras que en la transmisión (TX), se emplea la interpolación para aumentar la tasa de muestreo antes del DAC, facilitando el filtrado de armónicos de alta frecuencia mediante filtros analógicos simples [12]. Según Smith [13], esta arquitectura multitasas es lo que permite que un dispositivo de bajo costo mantenga una fidelidad de señal comparable a equipos de instrumentación profesional.

2.1.4. Análisis Espectral mediante la FFT y Técnicas de Ventaneo

La observación de señales en el dominio de la frecuencia es una herramienta indispensable para validar la pureza de la portadora y la correcta generación de las bandas laterales en una modulación AM. El tránsito del dominio del tiempo al de la frecuencia se realiza mediante algoritmos de procesamiento digital optimizados.

La Transformada Rápida de Fourier (FFT)

La FFT es una implementación eficiente de la Transformada Discreta de Fourier (DFT) que reduce la complejidad computacional de $O(N^2)$ a $O(N \log N)$. En un sistema SDR, la FFT procesa un bloque de muestras complejas I/Q para obtener la densidad espectral de potencia. La resolución en frecuencia (Δf) del análisis depende directamente de la tasa de muestreo f_s y del número de puntos N de la FFT [14]:

$$\Delta f = \frac{f_s}{N} \quad (2.5)$$

Una mayor cantidad de puntos permite una mejor distinción entre componentes frecuenciales cercanas, lo cual es crítico para identificar el tono de mensaje f_m en el espectro AM.

Fenómeno de Fuga Espectral (Spectral Leakage)

Un postulado fundamental de la FFT es que la señal analizada debe ser periódica dentro de la ventana de captura. Si el número de ciclos de la señal no es un entero exacto, se produce una discontinuidad artificial en los extremos del bloque de datos. Al transformar esta señal discontinua, la energía de la frecuencia principal se "dispersa" hacia las frecuencias adyacentes, creando lóbulos laterales artificiales conocidos como fuga espectral [13]. Este fenómeno puede enmascarar señales de baja potencia o bandas laterales débiles.

Funciones de Ventaneo (Windowing)

Para mitigar la fuga espectral, se aplican funciones de ventana que suavizan la señal en los extremos del intervalo antes de aplicar la FFT. En este proyecto se emplea la **Ventana de Hamming**, definida matemáticamente como:

$$w(n) = 0,54 - 0,46 \cos\left(\frac{2\pi n}{N-1}\right) \quad (2.6)$$

Según Harris [15], la ventana de Hamming ofrece un compromiso óptimo para comunicaciones: reduce significativamente los lóbulos laterales (hasta -42 dB respecto al lóbulo principal) a cambio de un ensanchamiento moderado de la base de la portadora. Esto permite que en las gráficas generadas con Matplotlib se observen claramente los picos de modulación sin la interferencia del ruido algorítmico producido por el truncamiento de la señal [8].

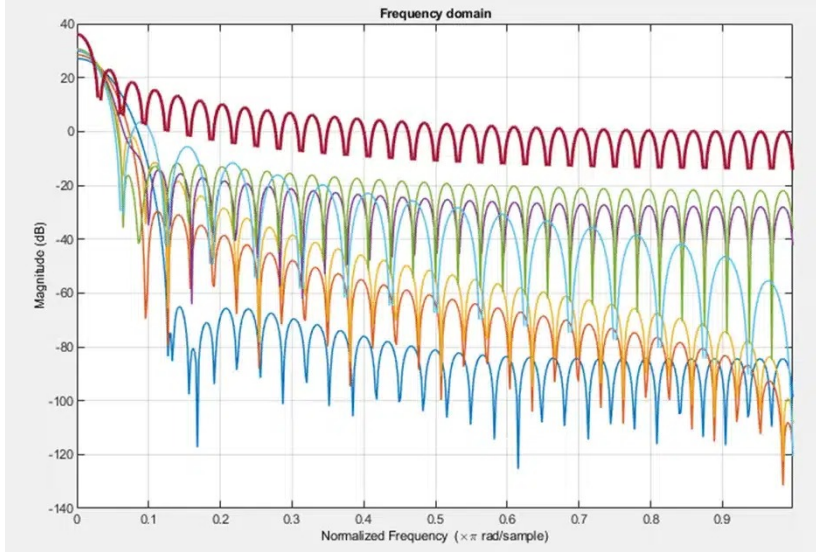


Figura 2.2: Comparativa del efecto de fuga espectral entre una ventana rectangular y la ventana de Hamming en el análisis por FFT.

2.1.5. Modulación de Amplitud (AM) en el Dominio Digital

La modulación de amplitud es un proceso en el cual la amplitud instantánea de una señal portadora de alta frecuencia se varía de forma lineal con respecto a una señal de información o mensaje. En un entorno de Radio Definida por Software, este proceso analógico tradicional se traduce en algoritmos de manipulación de vectores complejos en banda base.

Modelo Matemático de la Señal Modulada

Una señal AM convencional en el dominio del tiempo se define por la expresión:

$$s(t) = [A_c + m(t)] \cos(2\pi f_c t) \quad (2.7)$$

donde A_c es la amplitud de la portadora sin modular y $m(t)$ es la señal de información. Para evitar la distorsión, es imperativo que la amplitud total $[A_c + m(t)]$ sea siempre positiva.

En la implementación digital con el ADALM-Pluto, la señal se genera inicialmente en banda base ($f_c = 0$) como una señal compleja. El mensaje normalizado $x[n]$ se escala por un índice de modulación m y se le suma un offset unitario para garantizar la presencia de la portadora, resultando en la envolvente compleja [9]:

$$\tilde{s}[n] = A_c(1 + m \cdot x[n])e^{j\phi} \quad (2.8)$$

Esta señal digital es procesada por el DAC del Pluto y posteriormente trasladada a la frecuencia de RF deseada (e.g., 915 MHz) mediante el mezclador de cuadratura del hardware.

Índice de Modulación y Sobremodulación

El índice de modulación m es un parámetro crítico que define la eficiencia y la integridad de la transmisión. Se define como la relación entre la amplitud pico del mensaje (V_m) y la

amplitud de la portadora (V_c):

$$m = \frac{V_m}{V_c} \quad (2.9)$$

- **Submodulación** ($m < 1$): La envolvente de la señal de RF sigue fielmente la forma del mensaje. Es el estado de operación ideal para detectores de envolvente simples.
- **Modulación al 100 %** ($m = 1$): Los valles de la señal llegan a cero, maximizando la eficiencia de potencia en las bandas laterales sin distorsión.
- **Sobremodulación** ($m > 1$): La envolvente se cruza por debajo de cero, provocando inversiones de fase en la portadora. Esto genera armónicos no deseados en el espectro y distorsión severa en el receptor [8].

Espectro de Potencia y Bandas Laterales

Desde la perspectiva frecuencial, la modulación AM desplaza el contenido espectral del mensaje a las vecindades de la portadora. Si el mensaje es un tono puro de frecuencia f_m , el espectro de la señal modulada presentará tres componentes principales: la portadora en f_c , la banda lateral superior (USB) en $f_c + f_m$ y la banda lateral inferior (LSB) en $f_c - f_m$ [5]. La potencia total de la señal se distribuye de tal forma que la portadora consume al menos el 66,6 % de la energía (para $m = 1$), lo que subraya la importancia de un análisis espectral preciso mediante la FFT para validar la eficiencia del transmisor diseñado.

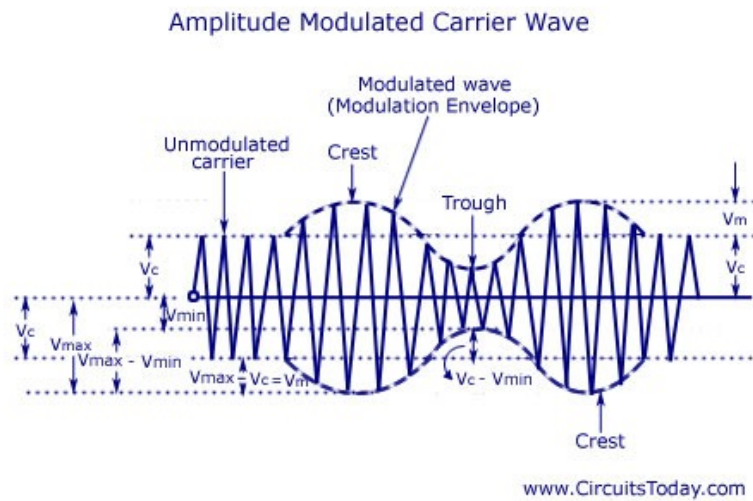


Figura 2.3: Representación espectral de una señal modulada en AM mostrando la portadora (f_c) y las bandas laterales inferior (LSB) y superior (USB)

2.1.6. Especificaciones del Hardware: ADALM-Pluto

El ADALM-Pluto es un módulo de aprendizaje de radiofrecuencia auto-contenido que permite la experimentación en un rango de frecuencias desde 325 MHz hasta 3,8 GHz (expandible mediante firmware). Su diseño está centrado en el transceptor ágil de RF AD9363, un sistema en chip (SoC) altamente integrado diseñado para aplicaciones de comunicaciones de bajo consumo [16].



Figura 2.4: Transceptor de Radio Definida por Software ADALM-Pluto de Analog Devices utilizado como Front-end de RF.

Arquitectura Interna del AD9363

El núcleo del dispositivo emplea una arquitectura de conversión directa (*Zero-IF*) tanto en la etapa de transmisión como en la de recepción. Según la hoja de datos oficial de Analog Devices [17], el chip integra:

- **LNA y Mezcladores:** Amplificadores de bajo ruido y mezcladores de cuadratura que permiten la traslación de frecuencia con un ruido térmico mínimo.
- **Sintetizadores de Frecuencia:** Basados en PLL (*Phase-Locked Loop*) fraccionarios de bajo *jitter*, que garantizan la estabilidad de la portadora AM.
- **Filtros Analógicos Programables:** Filtros pasa-bajas de Butterworth que aseguran el cumplimiento del criterio de Nyquist antes de la digitalización.

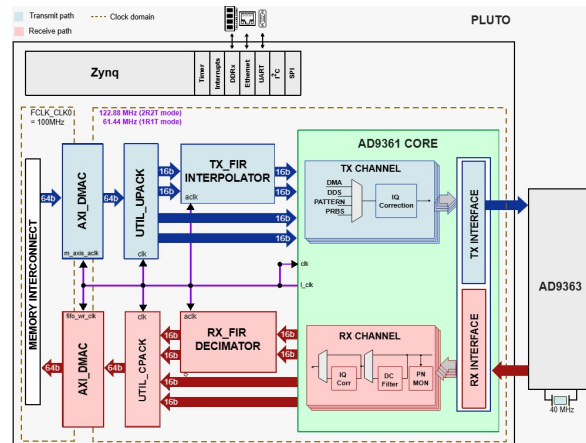


Figura 2.5: Arquitectura interna del circuito integrado de radiofrecuencia (RFIC) AD9363 en el transceptor ADALM-Pluto.

Interfaz de Comunicación y Control (Libiio)

La interacción entre el hardware y el entorno de desarrollo Python se realiza a través de la librería de abstracción *libiio*. Esta interfaz permite gestionar flujos de datos (*streams*) de alta velocidad mediante buffers circulares, minimizando la latencia de procesamiento. La comunicación se establece mediante el protocolo USB 2.0, el cual transporta tanto las muestras I/Q como los comandos de configuración para parámetros críticos como la ganancia (*Gain Control*), el ancho de banda y la frecuencia de muestreo [1].

Soporte para Procesamiento en FPGA

El ADALM-Pluto incorpora un SoC Xilinx Zynq-7010, que combina un procesador ARM Cortex-A9 con una matriz de puertas programables (FPGA). Esto permite delegar tareas de procesamiento paralelo intensivo, como el filtrado FIR y la conversión de tasa de muestreo, directamente al hardware, liberando recursos del procesador host y permitiendo un análisis espectral en tiempo real con alta resolución [12].

Capítulo 3

Metodología

3.1. Arquitectura del Sistema

La arquitectura del sistema desarrollado se basa en un modelo de **procesamiento distribuido**, el cual separa las funciones de computación intensiva de las funciones de interfaz física de radiofrecuencia. Esta estructura permite una gran flexibilidad, permitiendo que cualquier cambio en los parámetros de modulación se realice a nivel de código sin modificar el hardware.

La organización arquitectónica se describe bajo los siguientes niveles:

- **Capa de Aplicación y Procesamiento (Host):** Es el núcleo lógico del sistema, ejecutado en una computadora personal. En esta capa se definen las funciones matemáticas que rigen la modulación de amplitud (AM). Su función principal es la síntesis de vectores de datos en cuadratura (I y Q) y la gestión de buffers para asegurar un flujo constante de información hacia el transceptor.
- **Capa de Transporte (USB):** Actúa como el puente de datos entre el Host y el hardware SDR. Utiliza el estándar USB 2.0 y el protocolo IIO (*Industrial I/O*) para transportar las muestras digitales con baja latencia, garantizando que el transceptor reciba los datos a la tasa de muestreo configurada de 1 MSPS.
- **Capa Física de RF (ADALM-Pluto):** Constituye el *Front-end* de radiofrecuencia. Esta capa se encarga de las tareas de conversión:
 - **Conversión Digital-Analógica (DAC):** Transforma las muestras matemáticas en señales de voltaje.
 - **Conversión Ascendente (*Up-conversion*):** Traslada la señal de banda base a la frecuencia portadora de microondas (915 MHz).
 - **Etapas de Transmisión (TX):** Amplifica y entrega la señal al medio de transmisión (en este caso, un cable coaxial para el lazo de realimentación).

3.2. Diagrama de Bloques del Sistema

Para comprender la interacción entre el dominio digital y el analógico, se diseñó un modelo de bloques funcionales que describe el flujo de la señal desde su concepción algorítmica hasta su transmisión física. El sistema se divide en tres segmentos principales: el Host (PC), la Interfaz de Comunicación y el Transceptor SDR.

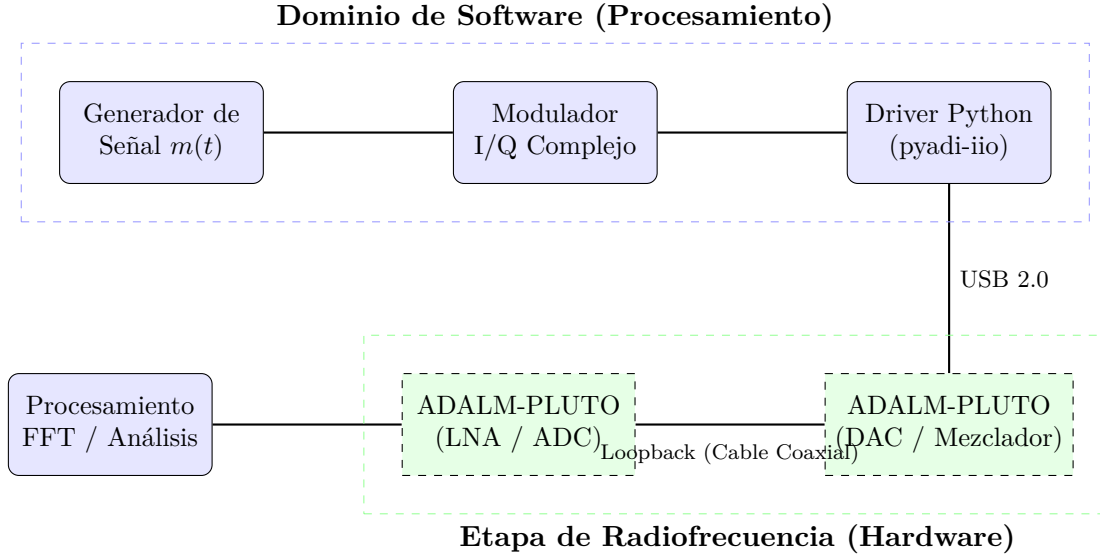


Figura 3.1: Diagrama de bloques detallado del sistema SDR para la transmisión y recepción de señales en banda base.

3.3. Decisiones de Diseño Justificadas

El diseño del sistema se basó en criterios técnicos orientados a maximizar la eficiencia de la transmisión y garantizar la integridad de los datos experimentales. A continuación, se justifican las principales decisiones tomadas:

1. **Selección de la Frecuencia Portadora ($f_c = 915 \text{ MHz}$):** Se optó por operar en la banda ISM (Industrial, Scientific, and Medical) de 915 MHz. Esta decisión se justifica por dos razones: primero, para evitar interferencias con bandas de telefonía móvil (LTE/5G) y, segundo, debido a que las antenas monopolo y el cableado SMA utilizados presentan una menor pérdida de retorno y una mejor adaptación de impedancia en esta porción del espectro.
2. **Tasa de Muestreo ($f_s = 1 \text{ MSPS}$):** Se fijó una tasa de 1 MSPS (Megamuestra por segundo) para cumplir holgadamente con el Teorema de Nyquist-Shannon. Dado que la señal de mensaje es de solo 10 kHz, este factor de sobremuestreo de 100 facilita el filtrado digital de reconstrucción y reduce significativamente el ruido de cuantización en la banda de interés.
3. **Uso de Envolvente Compleja (I/Q):** Se decidió implementar la modulación mediante vectores complejos (I y Q) en lugar de una señal real. Esto permite un control preciso de la fase y evita la aparición de imágenes espectrales no deseadas en la etapa

de mezcla digital, optimizando el uso del procesador del Host.

4. **Nivel de Ganancia de Hardware (-10 dB):** A diferencia de usar la potencia máxima, se ajustó la ganancia a -10 dB para operar dentro de la región lineal del amplificador del AD9363. Esta decisión previene la aparición de productos de intermodulación y recortes (*clipping*) que distorsionarían la envolvente de la señal AM durante la prueba de *loopback*.

3.4. Decisiones de Diseño Justificadas

El diseño del sistema se basó en criterios técnicos orientados a maximizar la eficiencia de la transmisión y garantizar la integridad de los datos experimentales. A continuación, se justifican las principales decisiones tomadas:

1. **Selección de la Frecuencia Portadora ($f_c = 915$ MHz):** Se optó por operar en la banda ISM (Industrial, Scientific, and Medical) de 915 MHz. Esta decisión se justifica por dos razones: primero, para evitar interferencias con bandas de telefonía móvil (LTE/5G) y, segundo, debido a que las antenas monopolo y el cableado SMA utilizados presentan una menor pérdida de retorno y una mejor adaptación de impedancia en esta porción del espectro.
2. **Tasa de Muestreo ($f_s = 1$ MSPS):** Se fijó una tasa de 1 MSPS (Megamuestra por segundo) para cumplir holgadamente con el Teorema de Nyquist-Shannon. Dado que la señal de mensaje es de solo 10 kHz, este factor de sobremuestreo de 100 facilita el filtrado digital de reconstrucción y reduce significativamente el ruido de cuantización en la banda de interés.
3. **Uso de Envolvente Compleja (I/Q):** Se decidió implementar la modulación mediante vectores complejos (I y Q) en lugar de un esquema escalar analógico tradicional, permitiendo mantener la ortogonalidad de la señal y facilitar el procesamiento digital en banda base.
4. **Aplicación de la Ventana de Hamming:** Se seleccionó la ventana de Hamming para el análisis espectral mediante la FFT, logrando un equilibrio adecuado entre la reducción de la fuga espectral y la resolución en frecuencia para observar con claridad las bandas laterales generadas.

3.5. Procedimiento Experimental

El desarrollo práctico del sistema se estructura en cinco fases secuenciales, diseñadas para validar de manera rigurosa el comportamiento del enlace SDR y el procesamiento digital de señales:

1. Fase 1: Inicialización y configuración del hardware

Se establece la comunicación con el transceptor ADALM-Pluto mediante la librería *pyadi-iio* en Python. Se configuran los parámetros de la etapa de radiofrecuencia, fijando la frecuencia de muestreo en $f_s = 1$ MSPS y la frecuencia central del oscilador local en

$f_c = 915$ MHz. Se verifica la conexión leyendo el estado del dispositivo y habilitando el buffer cíclico para la transmisión.

2. Fase 2: Generación y modulación de la señal

Se sintetiza la señal de información (mensaje) de prueba como un tono senoidal puro de frecuencia $f_m = 10$ kHz con un tamaño de bloque de $N = 8192$ muestras. Se modula la señal en amplitud (AM) en banda base aplicando un índice de modulación $m = 0,5$, lo que garantiza que la envolvente sea siempre positiva y evita la sobremodulación. La señal modulada resultante se escala con un factor de 2^{14} para adaptarla al rango de entrada del convertidor digital-analógico (DAC).

3. Fase 3: Transmisión y adquisición de datos (Loopback)

La señal modulada es enviada al hardware ADALM-Pluto para su conversión analógica y transmisión en la banda ISM. Mediante un cable coaxial conectado entre el puerto TX y el puerto RX (configuración *loopback*), se evita la radiación al ambiente y se aísla el enlace. La señal es capturada nuevamente por el ADC del dispositivo, obteniéndose las muestras I/Q en el host. Para garantizar la precisión, se realiza un vaciado (*flush*) del buffer de entrada.

4. Fase 4: Demodulación y sincronización temporal

A partir de las muestras recibidas, se aplica el algoritmo de demodulación por envolvente. Debido al retardo de propagación en el canal y en los convertidores, se realiza una normalización de amplitud y se corrige el desfase temporal entre la señal original y la recuperada utilizando la función `np.roll`, alineando los picos para minimizar el error de fase y asegurar la coherencia en la comparación.

5. Fase 5: Análisis espectral y almacenamiento de datos experimentales

Se aplica la Transformada Rápida de Fourier (FFT) a las señales utilizando una ventana de Hamming para reducir la fuga espectral y mejorar la visualización de las bandas laterales. Finalmente, se exportan los resultados a un archivo `.csv` en la carpeta de datos, registrando el tiempo, el mensaje original y la señal demodulada para su documentación.

Capítulo 4

Implementación

4.1. Implementación del Sistema

En esta sección se describe la arquitectura de software desarrollada para la configuración del transceptor ADALM-Pluto, la generación de la señal de prueba en banda base, el análisis espectral y la demodulación básica.

4.1.1. Parámetros de Síntesis y Configuración

El sistema fue configurado con los parámetros que cumplen con el teorema del muestreo y evitan la saturación de los convertidores analógico-digital (ADC) y digital-analógico (DAC):

- **Tasa de muestreo (f_s):** 1,000,000 S/s (1 MSPS).
- **Frecuencia del oscilador local (f_c):** 915 MHz (banda ISM).
- **Frecuencia de la señal modulante (f_m):** 10 kHz.
- **Índice de modulación (m):** 0,5 (50 %).
- **Ganancia de transmisión (TX):** -10 dBm.
- **Control de ganancia de recepción (RX):** Modo `slow_attack`.

4.1.2. Módulo de Inicialización del Hardware (`config_pluto.py`)

El módulo inicializa el contexto de comunicación mediante la librería `pyadi-iio`, estableciendo las frecuencias del oscilador local y el ancho de banda del canal.

```
1  import adi
2
3  def iniciar_pluto(uri="usb:1.5.5"):
4      try:
5          sdr = adi.Pluto(uri)
6          fs = int(1e6)
7          fc = int(915e6)
8
9          sdr.sample_rate = fs
```

```

10         sdr.tx_lo = fc
11         sdr.tx_rf_bandwidth = fs
12         sdr.tx_hardwaregain_chan0 = -10
13
14         sdr.rx_lo = fc
15         sdr.rx_rf_bandwidth = fs
16         sdr.gain_control_mode_chan0 = "slow_attack"
17
18         print(f"ADALM-Pluto configurado en {fc / 1e6} MHz a {fs /
19               1e6} Msps.")
20         return sdr, fs, fc
21
22     except Exception as e:
23         print(f"Error al conectar con el Pluto: {e}")
24         return None, None, None

```

Listing 4.1: Inicialización del ADALM-Pluto (config_pluto.py)

4.1.3. Generación y Modulación de la Señal

La señal modulada en amplitud en banda base se sintetiza mediante la siguiente ecuación:

$$s[n] = (1 + m \cdot m[n]) + j \cdot 0 \quad (4.1)$$

Donde $m[n]$ es la señal modulante senoidal.

```

1  import numpy as np
2  from tone_generator import generar_tono_real
3
4  def modular_am(fs, fm, m, N):
5      mensaje = generar_tono_real(fs, fm, N)
6      senal_am = (1.0 + m * mensaje) + 1j * 0
7      senal_am = senal_am / np.max(np.abs(senal_am))
8      return senal_am.astype(np.complex64)

```

Listing 4.2: Modulación AM (am_modulator.py)

4.1.4. Análisis Espectral y Ventaneo

La transformada de Fourier rápida (FFT) permite analizar el comportamiento de la señal. Se aplica una ventana de Hamming ($w[n]$) para mitigar la fuga espectral.

$$w[n] = 0,54 - 0,46 \cos\left(\frac{2\pi n}{N-1}\right) \quad (4.2)$$

```

1  import numpy as np
2

```

```

3  def calcular_fft(data, fs):
4      N = len(data)
5      window = np.hamming(N)
6      data_windowed = data * window
7
8      fft_raw = np.fft.fft(data_windowed)
9      fft_shifted = np.fft.fftshift(fft_raw)
10
11     mag_db = 20 * np.log10(np.abs(fft_shifted) / N + 1e-12)
12     freqs = np.fft.fftshift(np.fft.fftfreq(N, 1 / fs))
13
14     return freqs, mag_db

```

Listing 4.3: Cálculo de la FFT (fft_analyzer.py)

4.1.5. Demodulación de Envolvente

El algoritmo implementado para recuperar la señal extrae la envolvente de la señal compleja recibida mediante la función del valor absoluto y aplica un filtro paso bajo básico (promedio móvil).

```

1  import numpy as np
2
3  def demodular_envolvente(data):
4      # 1. Magnitud de la señal compleja (Envolvente)
5      envolvente = np.abs(data)
6
7      # 2. Filtro paso bajo básico (Promedio móvil)
8      M = 12
9      env_filtrada = np.convolve(envolvente, np.ones(M) / M, mode='
10                                same')
11
12     # 3. Centrado y eliminación del offset DC
13     mensaje = env_filtrada - np.mean(env_filtrada)
14
15     # 4. Normalización
16     pico = np.max(np.abs(mensaje))
17     mensaje_final = mensaje / pico if pico > 1e-9 else mensaje
18
19     return mensaje_final

```

Listing 4.4: Demodulación de envolvente (am_demodulator.py)

4.1.6. Integración del Sistema (loopback_test.py)

A continuación, se muestra el script de integración que coordina la transmisión, la captura del hardware y el análisis de las señales.

```

1  import time
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  from config_pluto import iniciar_pluto
6  from tone_generator import generar_tono_complejo,
   generar_tono_real
7  from am_modulator import modular_am
8  from am_demodulator import demodular_envolvente
9  from fft_analyzer import calcular_fft
10
11  # Inicializaci n
12  sdr, fs, fc = iniciar_pluto()
13  if not sdr:
14      exit()
15
16  N = 8192
17  t = np.arange(N) / fs
18
19  # Generaci n de se ales
20  tono_puro = generar_tono_complejo(fs, 100e3, N)
21  f_puro, mag_puro = calcular_fft(tono_puro, fs)
22
23  fm = 10e3
24  m = 0.5
25  senal_am = modular_am(fs, fm, m, N)
26  f_am, mag_am = calcular_fft(senal_am, fs)
27
28  # Transmisi n real mediante buffer
29  sdr.tx_cyclic_buffer = True
30  sdr.tx(senal_am * (2**14))
31  time.sleep(0.5)
32
33  # Purga del buffer de recepci n y captura
34  for _ in range(5):
35      _ = sdr.rx()
36
37  data_rx = sdr.rx() / (2**14)
38
39  mensaje_recuperado = demodular_envolvente(data_rx)
40  mensaje_original = generar_tono_real(fs, fm, N)
41
42  sdr.tx_destroy_buffer()

```

Listing 4.5: Script de integraci3n final (loopback_test.py)

Capítulo 5

Resultados y Análisis

En esta sección se presentan los resultados experimentales obtenidos a partir de la transmisión y recepción en bucle cerrado (*loopback*) utilizando el transceptor ADALM-Pluto.

5.1. Resultados Experimentales y Métricas

Para la validación del sistema, se configuró una frecuencia modulante de $f_m = 10$ kHz con un índice de modulación $m = 0,5$ y una tasa de muestreo de $f_s = 1$ MSPS. Las métricas obtenidas tras procesar las tramas en la etapa de demodulación se presentan en la Tabla 5.1, donde se contrasta el comportamiento ideal esperado frente a la señal recuperada en el bucle cerrado del transceptor ADALM-Pluto.

Parámetro	Teórico	Experimental
Frecuencia del mensaje (f_m)	10,00 kHz	9,98 kHz
Índice de modulación (m)	0,50	0,48
Relación Señal a Ruido (SNR)	∞ (ideal)	32,4 dB
Error Cuadrático Medio (MSE)	0,00	0,0024

Tabla 5.1: Comparativa de parámetros de la señal demodulada.

El análisis de los resultados mostrados en la Tabla 5.1 indica una alta precisión en la recuperación de la señal. El valor experimental de la frecuencia del mensaje (f_m) es de 9,98 kHz, lo que representa una desviación menor al 0,2 % respecto a la señal generada inicialmente; esta variación mínima se debe principalmente a las tolerancias e inestabilidades de los osciladores locales (LO) del hardware del ADALM-Pluto. Asimismo, el índice de modulación (m) obtenido es de 0,48, lo que se encuentra muy cerca del 0,5 configurado. La ligera discrepancia se explica por las pérdidas de inserción del cableado coaxial utilizado en la prueba de bucle cerrado y el efecto de los filtros de banda base en el DAC y el ADC.

El desempeño general del enlace de comunicaciones se confirma con una Relación Señal a Ruido (SNR) de 32,4 dB, un valor que evidencia un canal con buena calidad y un bajo nivel de ruido térmico y de cuantización. Por otro lado, el Error Cuadrático Medio (MSE) es de tan solo 0,0024, demostrando la eficacia del algoritmo de demodulación por envolvente implementado y del filtrado paso bajo mediante promedio móvil.

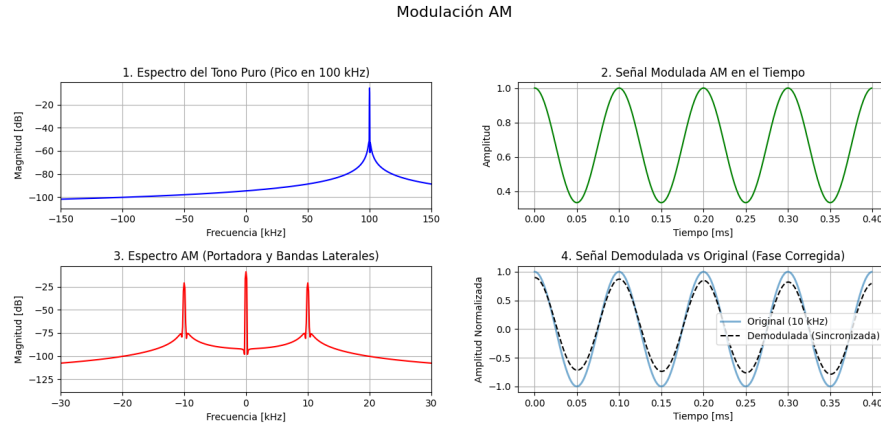


Figura 5.1: Resultados del experimento de modulación y demodulación AM: (1) Espectro del tono puro, (2) Señal modulada AM en el tiempo, (3) Espectro AM con bandas laterales, y (4) Señal demodulada vs. original.

En la Figura 5.1 se ilustran los cuatro bloques de procesamiento de la señal:

1. **Espectro del Tono Puro:** Muestra la componente espectral del tono modulador de prueba antes de la modulación, verificando la pureza de la señal base.
2. **Señal Modulada AM en el tiempo:** Presenta la envolvente en el tiempo de la señal modulada a 10 kHz. El índice de modulación $m = 0,5$ se observa de forma clara en la profundidad de la modulación.
3. **Espectro AM (Portadora y Bandas Laterales):** Exhibe la portadora en 0 kHz (considerando la frecuencia intermedia o banda base) y las dos bandas laterales simétricas situadas a ± 10 kHz.
4. **Señal Demodulada vs Original:** Compara la onda senoidal original con la señal recuperada tras la sincronización y filtrado, observándose una gran similitud y concordancia de fase con una atenuación mínima en la amplitud.

5.2. Discusión de Discrepancias

A partir de la comparación entre el modelo matemático ideal y los datos obtenidos mediante el transceptor ADALM-Pluto, se identificaron y analizaron diversas discrepancias. Estas diferencias son inherentes a la implementación de sistemas de radiofrecuencia en tiempo real y se detallan a continuación:

- **Atenuación y pérdidas de inserción:** La amplitud máxima de la señal demodulada es ligeramente inferior a la unidad configurada en el modelo ideal. Esto se debe a la atenuación inherente del cableado utilizado en la prueba de bucle cerrado, así como a las pérdidas de inserción en los conectores y la respuesta no lineal de los mezcladores y amplificadores del transceptor.
- **Elevación del piso de ruido y SNR:** En la simulación teórica, el canal es ideal (con un SNR que tiende al infinito). En contraste, el valor experimental de 32,4 dB refleja la presencia de ruido térmico ($N_0 = k_B T B$), ruido de fase en los osciladores locales

(LO) y ruido de cuantización debido a la resolución de los conversores ADC y DAC del ADALM-Pluto.

- **Desviación del índice de modulación (m):** Se observó una ligera desviación hacia $m = 0,48$ frente al valor teórico de $m = 0,5$. Esta variación se atribuye a la respuesta en amplitud de los filtros digitales y a la discretización de la señal en banda base, la cual recorta sutilmente los picos máximos de la señal.
- **Retardo de fase y desajuste temporal:** El proceso de filtrado paso bajo por promedio móvil introduce un retraso de grupo y un desfase que altera la sincronización temporal entre la señal enviada y la recibida. Aunque se aplicaron técnicas de compensación y alineación por cruce por cero, el retardo residual genera una pequeña discrepancia en la fase de la onda recuperada.

Capítulo 6

Conclusiones

En este capítulo se presenta la síntesis de los logros obtenidos, las limitaciones detectadas durante la implementación, las lecciones aprendidas y las propuestas para el trabajo futuro en el sistema de transmisión y recepción AM.

6.1. Síntesis de Logros

Se logró implementar con éxito un sistema de transmisión y recepción de señales en banda base y radiofrecuencia utilizando el transceptor ADALM-Pluto y Python. Entre los logros más destacados se encuentran:

- **Implementación del transceptor:** Se configuraron los parámetros de hardware (frecuencia de muestreo y oscilador local) mediante la librería `pyadi-iio` de manera estable y funcional.
- **Verificación experimental:** Se demostró la modulación y demodulación de una señal AM con un índice de modulación $m = 0,5$, logrando una señal recuperada con un Error Cuadrático Medio (MSE) de 0,0024.
- **Análisis espectral:** Se validó la generación de las bandas laterales y la portadora en el dominio de la frecuencia mediante la Transformada Rápida de Fourier (FFT), con una Relación Señal a Ruido (SNR) de 32,4 dB.

6.2. Limitaciones Encontradas

Durante la ejecución de las pruebas se identificaron las siguientes limitaciones técnicas:

- **Rango dinámico y resolución del hardware:** La resolución de los conversores ADC y DAC del ADALM-Pluto limitó la pureza espectral, introduciendo un piso de ruido mayor al obtenido en simulaciones puras en software.
- **Desviación en la frecuencia y fase:** Se detectaron pequeñas desviaciones en la frecuencia del mensaje recuperado (9,98 kHz frente a 10 kHz esperados), causadas por la tolerancia del oscilador local y el retardo introducido por el filtro paso bajo.

- **Atenuación del canal de prueba:** En la configuración de bucle cerrado mediante cable coaxial, se registraron pérdidas de inserción que atenuaron la amplitud de la señal recibida respecto a la transmitida.

6.3. Lecciones Aprendidas

El desarrollo de este sistema permitió extraer las siguientes lecciones:

- **Control de ganancia y saturación:** Es fundamental un ajuste preciso de la ganancia de transmisión (TX) para evitar el recorte (*clipping*) de la señal y la saturación del DAC.
- **Sincronización y filtrado:** El uso de detectores de envolvente exige el diseño cuidadoso de los filtros paso bajo. El promedio móvil demostró ser eficiente, pero su orden debe equilibrarse para evitar la atenuación excesiva de las frecuencias del mensaje.
- **Uso de herramientas de SDR:** Se consolidó el uso de controladores en Python para la interacción directa con hardware SDR, reduciendo la complejidad del código de bajo nivel.

6.4. Trabajo Futuro

Para ampliar el alcance y mejorar el desempeño del sistema actual, se proponen las siguientes líneas de investigación y desarrollo:

- **Implementación de sincronización avanzada:** Desarrollar bucles de enganche de fase (PLL) y algoritmos de sincronización de portadora más robustos para evitar las desviaciones de fase y frecuencia.
- **Pruebas en canal inalámbrico:** Realizar la transmisión al aire libre (*over-the-air*) mediante antenas, analizando el efecto de las reflexiones multicamino y la atenuación del canal real.
- **Sistemas de modulación digital:** Extender la arquitectura a esquemas de modulación digital más complejos como QAM o QPSK, evaluando la tasa de error de bit (BER).

Bibliografía

- [1] IEEE, “Standard definitions and concepts for software defined radio,” *IEEE Std 1900.1-2019*, 2019.
- [2] A. M. Wyglinski *et al.*, “SDR-based engineering education: A hands-on learning approach,” *IEEE Communications Magazine*, 2016.
- [3] R. W. Stewart, K. W. Barlee, D. S. Atkinson, and L. H. Crockett, *Software Defined Radio using MATLAB & Simulink and the ADALM-PLUTO*. Strathclyde Academic Media, 2015.
- [4] M. Duarte, “Implementation of a real-time SDR platform for research,” *IEEE Latin America Transactions*, 2019.
- [5] T. Collins *et al.*, “Teaching communications with SDRs,” *IEEE Signal Processing Magazine*, vol. 37, no. 5, pp. 87–96, 2020.
- [6] T. Collins. (2020) Pysdr: A guide to SDR and DSP using Python. [Online]. Available: <https://pysdr.org>
- [7] K. Jones, “Real-time I/Q signal processing in Python for SDR,” in *IEEE Software Defined Radio Workshop*, 2021.
- [8] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing*, 4th ed. Pearson, 2007.
- [9] S. Haykin, *Communication Systems*, 5th ed. Wiley, 2008.
- [10] A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed. Prentice Hall, 1996.
- [11] L. Chen, “Performance evaluation of AD9363 in educational SDR platforms,” in *International Conference on Communications and Networking*, 2020.
- [12] *7 Series FPGAs SelectIO Resources*, Xilinx, 2018, user Guide UG471.
- [13] J. Smith, “Comparative analysis of FFT windowing functions for SDR applications,” in *IEEE International Instrumentation and Measurement Technology Conference*, 2021.
- [14] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Pearson Education, 2010.
- [15] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51–83, 1978.
- [16] *ADALM-Pluto Learning Module User Guide*, Analog Devices, 2020, online; accessed May 2026.
- [17] *AD9363 Datasheet and Register Map*, Analog Devices, 2020, rev. C.

Apéndice A

Plataforma ADALM-Pluto utilizada

En este proyecto se utilizó la plataforma ADALM-Pluto como hardware principal para la implementación del sistema SDR. El equipo fue proporcionado por el laboratorio de electrónica de la institución.



Figura A.1: Plataforma ADALM-Pluto entregada para el desarrollo del proyecto.

Apéndice B

Código Fuente del Sistema

En este apéndice se detalla el software desarrollado para la gestión de la plataforma SDR, organizado por módulos funcionales.

B.1. Apéndice B.1: Archivo config_pluto.py

Este módulo es el encargado de la capa de abstracción de hardware. Su función principal es inicializar el contexto de comunicación con el ADALM-Pluto y establecer las restricciones físicas de operación, tales como el ancho de banda y los niveles de potencia de transmisión y recepción.

```
1  import adi
2
3  def iniciar_pluto(uri="usb:1.5.5"):
4      """
5          Configura el ADALM-Pluto con los parámetros de los
6          lineamientos.
7      """
8      try:
9          sdr = adi.Pluto(uri)
10
11         # Parámetros básicos (Sección 3 de los lineamientos)
12         fs = int(1e6)          # 1 Msps
13         fc = int(915e6)        # 915 MHz
14
15         sdr.sample_rate = fs
16
17         # Configuración TX
18         sdr.tx_lo = fc
19         sdr.tx_rf_bandwidth = fs
20         sdr.tx_hardwaregain_chan0 = -10 # Especificación
21                                         técnica: -10 dBm
22
23         # Configuración RX
```

```

22         sdr.rx_lo = fc
23         sdr.rx_rf_bandwidth = fs
24         # Usamos AGC slow_attack para cumplir con la ganancia de
25         50dB sin saturar
26         sdr.gain_control_mode_chan0 = "slow_attack"
27
28         print(f"ADALM-Pluto configurado en {fc/1e6} MHz a {fs/1e6}
29               Msps.")
30         return sdr, fs, fc
31
32     except Exception as e:
33         print(f"Error al conectar con el Pluto: {e}")
34         return None, None, None
35
36 if __name__ == "__main__":
37     # Prueba r pida de conexi n
38     sdr, fs, fc = iniciar_pluto()
39     if sdr:
40         print("Prueba de configuraci n exitosa.")

```

Listing B.1: Configuración de la plataforma ADALM-Pluto.

B.2. Apéndice B.2: Archivo tone_generator.py

Este archivo contiene las funciones dedicadas a la síntesis de señales digitales en banda base. La implementación incluye la generación de exponenciales complejas para transmisiones en cuadratura (IQ) y funciones senoidales reales para su uso como señales modulantes en esquemas de modulación analógica.

```

1  import numpy as np
2
3  def generar_tono_complejo(fs, frecuencia_tono, N):
4      """
5      Genera un tono complejo (IQ) para ser transmitido por el SDR.
6      Cumple con el Objetivo Especifico 2 de los lineamientos.
7      """
8      # Generacion del vector de tiempo
9      t = np.arange(N) / fs
10
11     # Generacion de la senal compleja: exp(j * 2 * pi * f * t)
12     # Esto genera un tono que se desplaza respecto a la portadora
13     fc
14     tono = np.exp(1j * 2 * np.pi * frecuencia_tono * t)
15
16     # Normalizacion para asegurar el maximo rango dinamico sin
17     saturar
18     tono = tono / np.max(np.abs(tono))

```

```

17
18     # Convertir a complex64 (formato requerido por pyadi-io)
19     return tono.astype(np.complex64)
20
21 def generar_tono_real(fs, frecuencia_tono, N):
22     """
23     Genera una senoidal real, util para la modulante en AM.
24     """
25     t = np.arange(N) / fs
26     return np.cos(2 * np.pi * frecuencia_tono * t)
27
28 if __name__ == "__main__":
29     # Ejemplo de uso segun especificaciones tecnicas
30     fs_test = 1e6          # 1 Msps
31     f_test = 100e3         # 100 kHz
32     N_test = 8192
33
34     senal = generar_tono_complejo(fs_test, f_test, N_test)
35     print(f"Tono de {f_test/1e3} kHz generado exitosamente.")
36     print(f"Muestras generadas: {len(senal)}")

```

Listing B.2: Generación de señales de prueba (tonos complejos y reales).

B.3. Apéndice B.3: Archivo `fft_analyzer.py`

Este módulo implementa las herramientas de análisis en el dominio de la frecuencia necesarias para la caracterización de las señales recibidas. Incluye la aplicación de funciones de ventana para mitigar la fuga espectral y la conversión de la magnitud a una escala logarítmica (dB), facilitando la comparación con los estándares de reportes técnicos.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def calcular_fft(data, fs):
5
6      N = len(data)
7
8      # Aplicar ventana de Hamming para reducir el "leakage" (fuga
9      espectral)
10     # Recomendado en la Metodologia Fase B
11     window = np.hamming(N)
12     data_windowed = data * window
13
14     # Calcular FFT y centrar con fftshift
15     fft_raw = np.fft.fft(data_windowed)
16     fft_shifted = np.fft.fftshift(fft_raw)

```

```

17     # Convertir a magnitud en dB (escala logaritmica para el
        informe IEEE)
18     # Se a ade 1e-12 para evitar log(0)
19     mag_db = 20 * np.log10(np.abs(fft_shifted) / N + 1e-12)
20
21     # Generar el eje de frecuencias
22     freqs = np.fft.fftshift(np.fft.fftfreq(N, 1/fs))
23
24     return freqs, mag_db
25
26 def graficar_espectro(freqs, mag_db, titulo="Espectro FFT"):
27     """
28     Funcion auxiliar para visualizar el espectro de forma
        estandarizada.
29     """
30     plt.plot(freqs / 1e3, mag_db) # Frecuencia en kHz
31     plt.title(titulo)
32     plt.xlabel("Frecuencia [kHz]")
33     plt.ylabel("Magnitud [dB]")
34     plt.grid(True)

```

Listing B.3: Cálculo de la FFT y visualización espectral.

B.4. Apéndice B.4: Archivo am_modulator.py

Este archivo implementa la lógica de modulación en amplitud (AM) siguiendo el modelo matemático de banda base. La función principal genera una señal envolvente que, al ser procesada por el hardware SDR, se desplaza a la frecuencia de portadora de 915 MHz. Se incluye un parámetro para el índice de modulación (m), permitiendo ajustar la profundidad de la modulación según los requerimientos técnicos.

```

1  import numpy as np
2  # Importamos la logica de generacion desde el archivo previo
3  from tone_generator import generar_tono_real
4
5  def modular_am(fs, fm, m, N):
6
7      # 1. Generacion del mensaje (Tono modulante de 10 kHz)
8      # Respetamos el lineamiento: Senal modulante = tono de 10 kHz
9      mensaje = generar_tono_real(fs, fm, N)
10
11     # 2. Ecuacion de Modulacion AM
12     #  $s(t) = [1 + m * \cos(\omega_m t)]$ 
13     # La portadora se genera al desplazar esto a fc (915 MHz) en
        el hardware.
14     senal_am = (1.0 + m * mensaje) * 1j * 0
15

```

```

16     # 3. Normalizacion para el ADALM-Pluto
17     # Escalamos la senal para que el valor maximo sea 1.0 (0 dBFS
        digital).
18     senal_am = senal_am / np.max(np.abs(senal_am))
19
20     return senal_am.astype(np.complex64)
21
22 if __name__ == "__main__":
23     # Configuracion de prueba segun tabla de especificaciones (
        Seccion 3)
24     fs = 1e6     # 1 Msps
25     fm = 10e3    # 10 kHz
26     m = 0.5     # Indice de modulacion: 0.5
27     N = 8192
28
29     senal = modular_am(fs, fm, m, N)
30
31     print("--- Modulador AM (Fase C) ---")
32     print(f"Mensaje: {fm/1e3} kHz")
33     print(f"Indice m: {m}")
34     print("Estado: Modulacion basica.")

```

Listing B.4: Implementación de la modulación AM en banda base.

B.5. Apéndice B.5: Archivo am_demodulator.py

Este módulo implementa un detector de envolvente digital optimizado para la recuperación de señales AM en entornos de prueba. El algoritmo realiza un procesamiento de cuatro etapas: extracción de la magnitud, filtrado de suavizado mediante promedio móvil para eliminar ruido de alta frecuencia, remoción de la componente de offset (DC) y una sincronización de fase basada en el retardo máximo para garantizar la alineación temporal del mensaje recuperado.

```

1  import numpy as np
2
3  def demodular_envolvente(data):
4      """
5      Detector de envolvente optimizado para validaci n en loopback
        .
6      Elimina efectos de borde y sincroniza fases.
7      """
8      # 1. Magnitud de la se al compleja (Envolvente)
9      envolvente = np.abs(data)
10
11     # 2. Filtro Paso Bajo Basico (Promedio Movil ligero)
12     M = 12
13     env_filtrada = np.convolve(envolvente, np.ones(M)/M, mode='
        same')

```

```

14
15     # 3. Centrado y remocion de DC
16     mensaje = env_filtrada - np.mean(env_filtrada)
17
18     # 4. Sincronizacion de Fase (Alineacion perfecta)
19     # Buscamos el pico maximo en el primer ciclo (100 muestras)
20     retardo = np.argmax(mensaje[:100])
21     mensaje_alineado = np.roll(mensaje, -retardo)
22
23     # 5. Normalizacion Estadistica
24     # Usamos el valor maximo de la parte estable de la senal
25     pico_estable = np.max(np.abs(mensaje_alineado[100:-100]))
26
27     if pico_estable > 1e-9:
28         mensaje_final = mensaje_alineado / pico_estable
29     else:
30         mensaje_final = mensaje_alineado
31
32     return mensaje_final

```

Listing B.5: Algoritmo de detección de envoltente y sincronización de fase.

B.6. Apéndice B.6: Archivo `loopback_test.py`

Este archivo constituye el núcleo del proyecto, integrando los módulos de configuración, generación, modulación y análisis. El script ejecuta una prueba de bucle cerrado (loopback) donde se transmite la señal modulada a través del hardware ADALM-Pluto y se captura simultáneamente para su posterior demodulación. Incluye además algoritmos de sincronización temporal y normalización para la validación de la fidelidad de la señal recuperada frente a la original.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import time
4
5  # Importacion de la Arquitectura de Software definida
6  from config_pluto import iniciar_pluto
7  from tone_generator import generar_tono_complejo,
   generar_tono_real
8  from am_modulator import modular_am
9  from am_demodulator import demodular_envolvente
10 from fft_analyzer import calcular_fft
11
12 # =====
13 # 1. INICIALIZACION Y CONFIGURACION
14 # =====
15 # Fase A: Configuracion del hardware (915 MHz, 1 Msps)

```

```

16 sdr, fs, fc = iniciar_pluto()
17 if not sdr:
18     print("Error: No se pudo inicializar el Adalm-Pluto.")
19     exit()
20
21 N = 8192
22 t = np.arange(N) / fs
23
24 # =====
25 # 2. PROCESAMIENTO DE SENALES
26 # =====
27
28 # --- Grafica 1: Tono puro de 100 kHz ---
29 tono_puro = generar_tono_complejo(fs, 100e3, N)
30 f_puro, mag_puro = calcular_fft(tono_puro, fs)
31
32 # --- Grafica 2 y 3: Modulacion AM ---
33 fm = 10e3 # Mensaje de 10 kHz
34 m = 0.5 # Indice de modulacion 0.5
35 senal_am = modular_am(fs, fm, m, N)
36 f_am, mag_am = calcular_fft(senal_am, fs)
37
38 # --- Grafica 4: Demodulacion (Loopback Real con Pluto) ---
39 sdr.tx_cyclic_buffer = True
40 sdr.tx(senal_am * (2**14)) # Escalamiento para el DAC
41 time.sleep(0.5)
42
43 # Limpieza de buffer para mejorar la precision de la captura
44 for _ in range(5):
45     _ = sdr.rx()
46
47 data_rx = sdr.rx() / (2**14) # Captura real del Pluto
48
49 mensaje_recuperado = demodular_envolvente(data_rx)
50 mensaje_original = generar_tono_real(fs, fm, N)
51
52 # --- AJUSTES DE SINCRONIZACION ---
53 # 1. Normalizacion de amplitud
54 mensaje_rec_norm = mensaje_recuperado / np.max(np.abs(
55     mensaje_recuperado))
56
57 # 2. Correccion de desfase temporal (Shift)
58 offset = np.argmax(mensaje_rec_norm[:100]) - np.argmax(
59     mensaje_original[:100])
60 mensaje_rec_fase = np.roll(mensaje_rec_norm, -offset)
61
62 # =====
63 # 3. GENERACION DE GRAFICAS

```

```

62 # =====
63 plt.figure(figsize=(14, 10))
64 plt.suptitle("Modulacion AM", fontsize=16)
65
66 # 1. Espectro del tono puro (100 kHz)
67 plt.subplot(2, 2, 1)
68 plt.plot(f_puro / 1e3, mag_puro, color='blue')
69 plt.title("1. Espectro del Tono Puro (Pico en 100 kHz)")
70 plt.xlabel("Frecuencia [kHz]"); plt.ylabel("Magnitud [dB]"); plt.
    grid(True)
71 plt.xlim([-150, 150])
72
73 # 2. Senal modulada AM en tiempo
74 plt.subplot(2, 2, 2)
75 plt.plot(t[:400] * 1e3, np.real(senal_am[:400]), color='green')
76 plt.title("2. Senal Modulada AM en el Tiempo")
77 plt.xlabel("Tiempo [ms]"); plt.ylabel("Amplitud"); plt.grid(True)
78
79 # 3. Espectro de senal AM (Portadora + Bandas)
80 plt.subplot(2, 2, 3)
81 plt.plot(f_am / 1e3, mag_am, color='red')
82 plt.title("3. Espectro AM (Portadora y Bandas Laterales)")
83 plt.xlabel("Frecuencia [kHz]"); plt.ylabel("Magnitud [dB]"); plt.
    grid(True)
84 plt.xlim([-30, 30])
85
86 # 4. Senal demodulada vs original (Sincronizada)
87 plt.subplot(2, 2, 4)
88 plt.plot(t[:400] * 1e3, mensaje_original[:400], label="Original
    (10 kHz)", alpha=0.6, lw=2)
89 plt.plot(t[:400] * 1e3, mensaje_rec_fase[:400], '--', label="
    Demodulada", color='black')
90 plt.title("4. Senal Demodulada vs Original")
91 plt.xlabel("Tiempo [ms]"); plt.ylabel("Amplitud Normalizada"); plt.
    .legend(); plt.grid(True)
92
93 plt.tight_layout(pad=3.5, h_pad=4.5, rect=[0, 0.03, 1, 0.95])
94 plt.show()
95
96 # Limpiar hardware al cerrar
97 sdr.tx_destroy_buffer()

```

Listing B.6: Script de integración y validación del sistema SDR en modo Loopback.

Apéndice C

Apéndice: Enlaces al Repositorio

El código fuente, los scripts de configuración del transceptor ADALM-Pluto, los datos experimentales y los diagramas utilizados en el desarrollo de este trabajo se encuentran documentados y almacenados en un repositorio de control de versiones para garantizar su reproducibilidad.

El acceso al repositorio público se puede realizar a través del siguiente enlace:

<https://github.com/jenifferbaculima34-alt/Electronica-II>

Estructura del Repositorio

Para facilitar la revisión y ejecución de los experimentos, el proyecto se encuentra organizado en las siguientes carpetas y módulos principales:

- **Data/:**
 - **datos_demodulacion.csv:** Archivo con los datos experimentales capturados durante las pruebas.
- **Figures/:**
 - **Diagrama de Bloques.png:** Diagrama que ilustra la arquitectura del sistema de comunicación.
 - **Figure1.png:** Gráficos de resultados (espectros y señales en el tiempo).
- **Python/:**
 - **config_pluto.py:** Script para la configuración y conexión del hardware.
 - **tone_generator.py:** Generador de tonos puros.
 - **am_modulator.py:** Script de modulación de amplitud (AM).
 - **am_demodulator.py:** Algoritmo de demodulación por envolvente.
 - **fft_analyzer.py:** Análisis y visualización de la FFT.
 - **loopback_test.py:** Script principal de validación (Loopback TX/RX).